

Windows Compatibility and Scientific Computing

Rohit Goswami · 2026-02-16 · fortran · windows · build · science

Notes on cross-platform portability for scientific codes with legacy components.

Background

Most computational chemistry codes are developed on Linux. macOS shows up in CI matrices or in workshops, a [conda-forge feedstock](#) might cover the major platforms, but Windows tends to be tested last and debugged reluctantly [^fn:1]. This is simply a consequence of the platforms available in most research groups and the priorities of the people writing the code [^fn:2].

I have been maintaining [eOn](#) [^fn:3] for a while now, through a GPR-dimer implementation ([Goswami et al. 2025](#)), an on-the-go variant ([Goswami and Jónsson 2025](#)), the NEB-MMF work ([Goswami, Gunde, and Jónsson 2026](#)), my thesis ([Goswami 2025](#)), and more recently the [metatensor ecosystem integration](#) ([Bigi et al. 2026](#)) [^fn:4] and an [atomistic cookbook recipe](#) for NEB with machine learning potentials. Path diagnostics for those NEB runs live in [Visual Diagnostics for NEB Paths](#). Along the way, and through contributions to [chemfiles](#) and [asv](#), I have collected a tidy set of Windows-specific failure modes. Each has cost at least one CI cycle and occasionally an afternoon.

Reserved Filenames

Windows inherits from DOS a set of [reserved device names](#) that cannot be used as filenames, regardless of extension: `CON`, `PRN`, `AUX`, `NUL`, `COM1` through `COM9`, `LPT1` through `LPT9`. Case-insensitive, naturally. The [Microsoft documentation](#) is thorough, but in practice most people discover these constraints from a CI failure rather than from reading the docs.

`aux.py` in `asv_runner` (2023)

When writing [asv_runner](#), the runner component for the `asv` benchmarking tool, I had a perfectly reasonable file called `asv_runner/aux.py` for auxiliary benchmark configuration [^fn:5]. On Linux, no problem. On Windows:

```
ERROR: For req: asv-runner. The wheel has a file 'asv_runner/aux.py'
trying to install outside the target directory
```

The wheel could not even be *installed*, let alone the repository cloned [^fn:6]. Since `asv` is used by NumPy, SciPy, and friends for performance tracking, this surfaced quickly. The [fix](#) was to rename the file. The same issue appeared in [pypotlib](#) shortly after, because apparently one lesson is never enough.

`CON.cpp` in `chemfiles` (2026)

Three years later, while adding `.con` file support to [chemfiles](#), I named the implementation files `CON.hpp` and `CON.cpp` after the format they implement [^fn:7]. `CON` is of course also a reserved device name.

The files were renamed to `_CON.hpp` and `_CON.cpp`. The list of reserved names is short enough to memorize, but apparently long enough to forget between projects. The divergence in kernel handling is shown in [Figure 1](#).

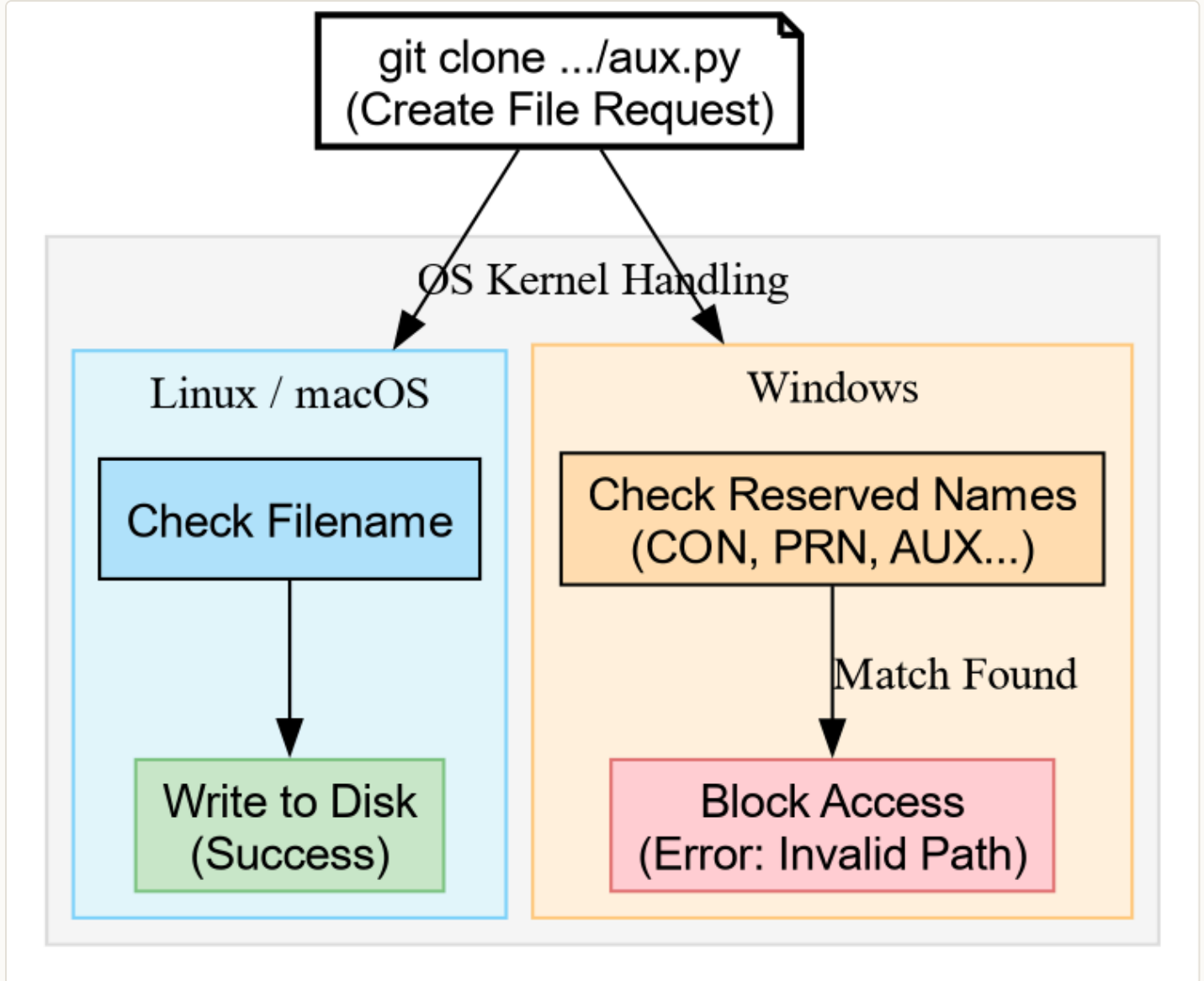


Figure 1: Linux writes 'aux.py'. Windows treats 'AUX' as a device and refuses the path before it reaches the disk.

Stack Sizes and Legacy Fortran

The default stack size on Linux is typically 8 MB (`ulimit -s`). On Windows, it is 1 MB. This is rarely a problem for modern C++ since `Eigen::MatrixXd` and friends allocate on the heap, but it matters enormously for legacy Fortran, where local arrays live on the stack by default.

The GAGAFE Subroutine

The eOn code includes Embedded Atom Model potentials implemented in Fortran 77. The core routine is called `GAGAFE` , and I rather enjoy knowing where the name comes from [^fn:8]:

GAGAFE stands for 'group of atoms, group of atoms, force and energy'. In a system with two types of atoms, A and B, GAGAFE needed to be called three times, once for the A-A interactions, then for the B-B interactions and finally for the A-B interactions. This is coming from the code written in the H. C. Andersen group at Stanford (starting around 1980 when he wrote his famous paper on the velocity Verlet algorithm and the isobaric simulations with the extended Lagrangian). The code ran on a PDP-11 computer in combination with an array processor. I was a post-doc in the group 1986-1988 and kept this basic structure in the F77 code I wrote when I moved to Seattle in 1988.

– Hannes Jónsson (private communication)

There is something appealing about a subroutine name with a provenance older than most of its users. The code has been running continuously for over four decades, and it still works.. except on Windows, where the local arrays blow the stack.

The subroutine declares arrays dimensioned with compile-time constants:

```
c parameters.cmn
  parameter (MAXPRS = 200000)

c gagafeDblexp.f
  DIMENSION phi(MAXPRS), phivirst(MAXPRS)
  DIMENSION RA1(MAXC00), RA2(MAXC00), FA1(MAXC00), FA2(MAXC00)
```

`phi` and `phivirst` alone consume around 3 MB. Including the remaining arrays, the total stack footprint is roughly around 3.6 MB which is well beyond the 1 MB Windows default. The client crashes on entry to the potential routine with exit code `3221225725` (decimal), which is `0xC00000FD`: `STATUS_STACK_OVERFLOW` (see Figure 2).

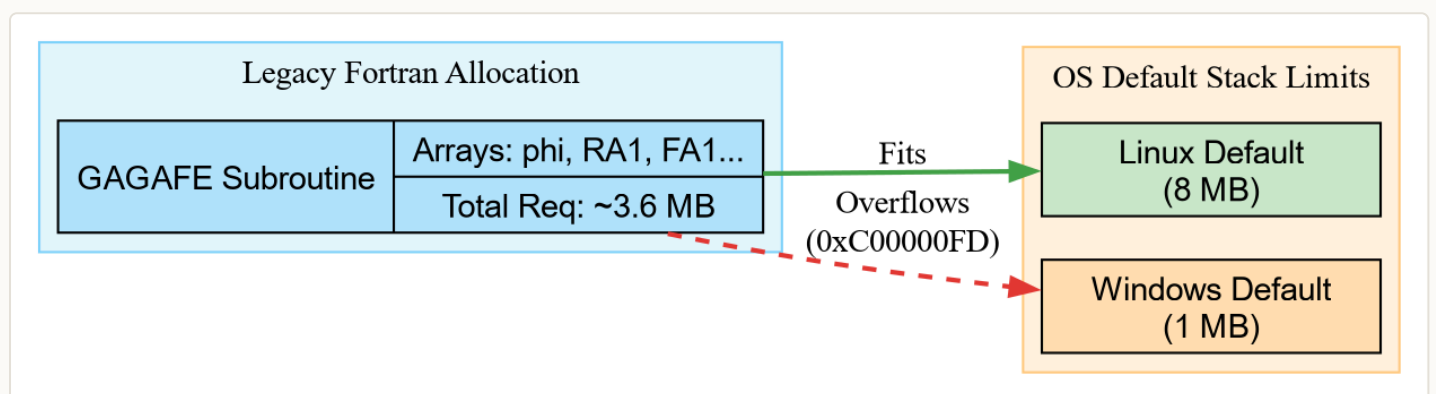


Figure 2: GAGAFE's 3.6 MB stack arrays fit Linux's 8 MB default and overflow Windows' 1 MB default.

The Fix

For Meson-based builds, the linker can request a larger stack:

```
if is_windows
  if is_mingw
    _linkargs += ['-Wl,--stack,16777216'] # 16 MB
  elif cppc.get_id() == 'msvc'
    _linkargs += ['/STACK:16777216']
  endif
endif
```

The alternative is refactoring the Fortran to use `ALLOCATABLE` arrays or `COMMON` blocks. For code with this kind of lineage, the linker flag is less invasive. More on working with inherited Fortran in general can be found in an [earlier post](#).

Stdout Redirection and `spdlog`

The `spdlog` logging library provides `stdout_color_sink_mt`, which uses ANSI escape codes on Linux and `WriteConsole` on Windows for colored output. The catch: `WriteConsole` [only works](#) when the output handle is an actual console [^fn:9]. When a parent process redirects stdout to a file as the eOn Python server does for every client job, `WriteConsole` fails silently.

The symptom is that the client produces no stdout whatsoever. stderr is empty. The exit code may or may not be informative depending on what else breaks downstream. The fix is to select the non-color sink:

```

#ifdef _WIN32
    auto console_sink =
        std::make_shared<spdlog::sinks::stdout_sink_mt>();
#else
    auto console_sink =
        std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
#endif

```

This took longer to diagnose than it should have, mostly because “no output” does not immediately suggest “color codes”, a mechanism detailed in Figure 3.

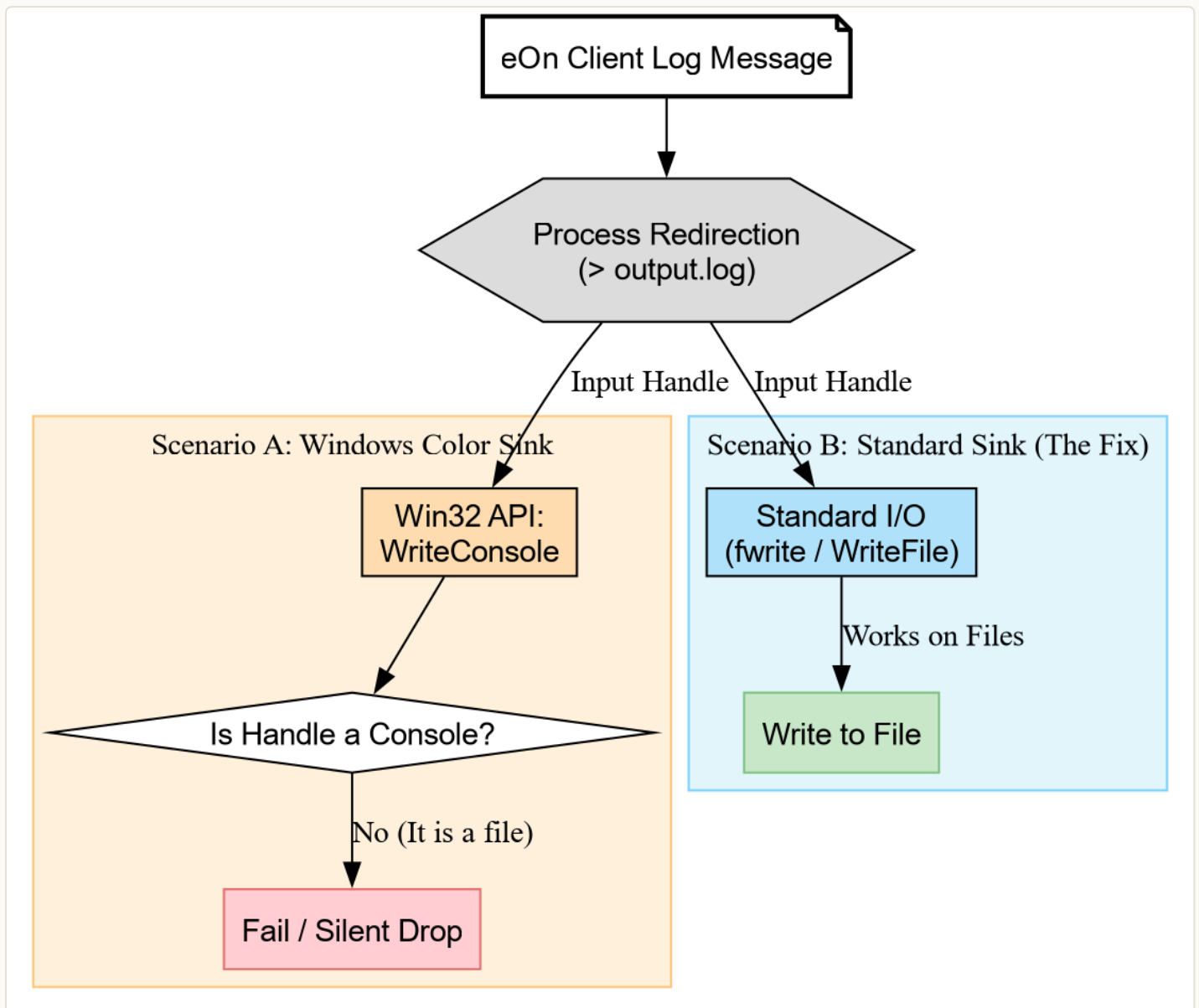


Figure 3: `spdlog`'s Windows color sink calls `WriteConsole`, which drops the line when stdout is a file.

This has **been fixed** in recent versions of `spdlog`, and was likely simply masking errors from the `gagafe` issue.

Quill macros and MSVC constexpr

`spdlog` left. `Quill` arrived with [eon-feedstock v2.12](#) / [eOn #327](#). The convenience wrappers were:

```

#define EONC_LOG_INFO(...) LOG_INFO(eonc::log::get(), __VA_ARGS__)

```

That expands into a `static constexpr MacroMetadata` initializer. The logger argument is only used at runtime. MSVC still tries to evaluate `eonc::log::get()` as part of the `constexpr` scope and dies with `C2440` / “a non-constant (sub-)expression was encountered” [^fn:10]. GCC and Clang ignore the unused

argument. The fix is a logger variable, not a function call in the macro argument list. Fourteen files.

Same class as `interface`: MSVC is stricter about a rule the other compilers treat as a suggestion.

The MinGW/MSVC ABI Boundary

On conda-forge, Fortran libraries are built with MinGW (`m2w64`) because there is no MSVC Fortran compiler. The entire grimme-lab stack (`xtb` , `tblite` , `dftd4` , `mctc-lib`) uses MinGW consistently, so internally there is no mismatch. The problem appears when a C++ project built with MSVC needs to link against one of these libraries.

This surfaced while enabling Windows builds for the [eOn v2.10.0](#) conda-forge [feedstock](#). eOn links against `libxtb` for the tight-binding potential, and against `libtorch` and `libmetatensor` for machine learning potentials. The latter two come from the MSVC ecosystem. There is no world in which everything uses the same compiler [^fn:11].

The Import Library Problem

MinGW produces `libxtb.dll.a` (a GNU-style import library). MSVC’s linker does not understand this format; it expects `xtb.lib`. The xtb conda package does not ship one because it has no reason to [^fn:12].

The workaround is to generate an MSVC-compatible import library from the DLL exports:

```
dumpbin /EXPORTS "%LIBRARY_BIN%\libxtb-6.dll" > xtb_exports.txt
echo LIBRARY libxtb-6.dll > xtb.def
echo EXPORTS >> xtb.def
for /f "skip=19 tokens=4" %%A in (xtb_exports.txt) do (
    if not "%%A"==" " echo %%A >> xtb.def
)
lib /DEF:xtb.def /OUT:"%LIBRARY_LIB%\xtb.lib" /MACHINE:X64
```

This works because `xtb` exposes a pure C API (`xtb.h`), and the C ABI is compatible between MinGW and MSVC. C++ name mangling would be fatal here, but `xtb` [^fn:13] provide a C interface. The dependency chain is illustrated in Figure 4.

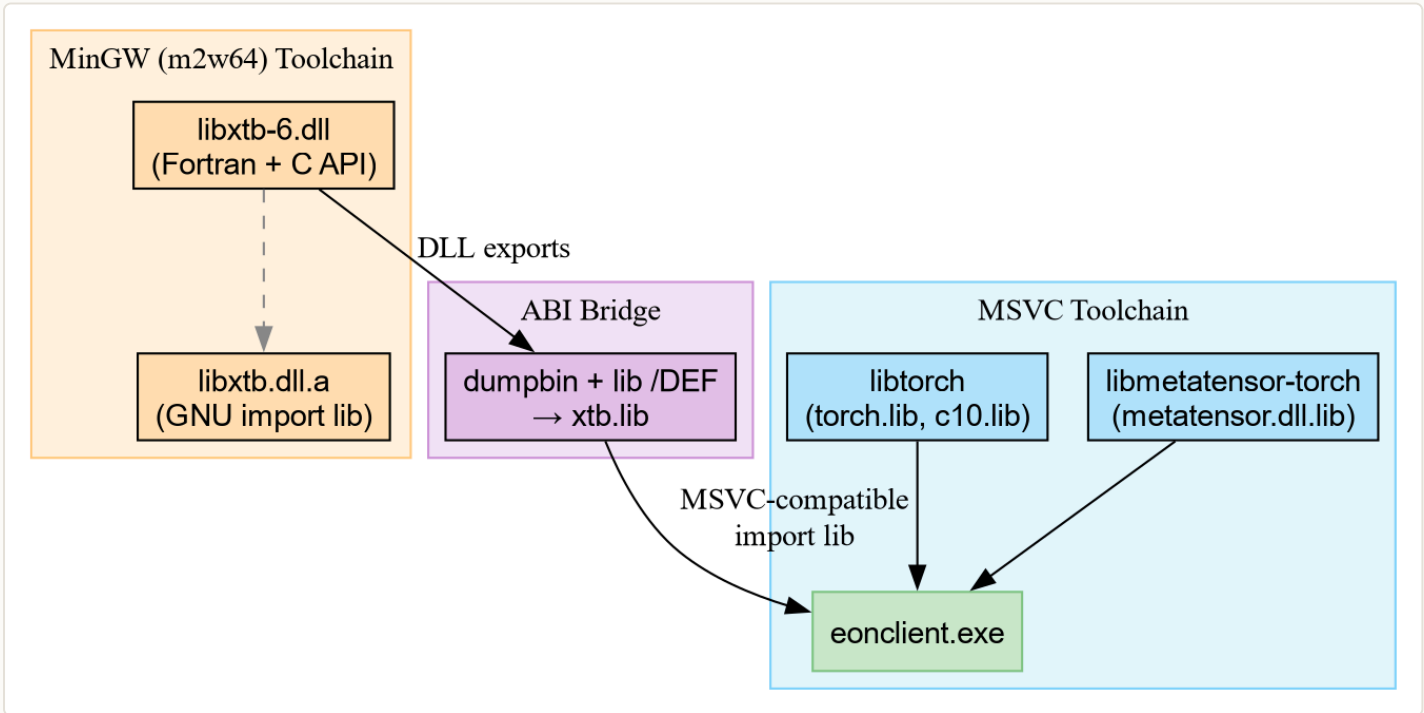


Figure 4: ‘dumpbin’ plus ‘lib /DEF’ turns a MinGW ‘libxtb’ DLL into an MSVC import library so ‘eonclient’ can link both toolchains.

find_library and Missing Import Libraries

A related pattern appears in the `libtorch` integration from the same [feedstock PR](#). The meson build finds torch libraries with `find_library`, which defaults to `required: true`:

```
lib_torch_list = ['c10', 'torch', 'torch_cpu', 'torch_global_deps']
tdeps = []
foreach lib_name : lib_torch_list
  tdeps += cpc.find_library(lib_name, dirs: [LIB_TORCH_LIB_PATH])
endforeach
```

On Windows, `torch_global_deps` ships as a DLL with no corresponding `.lib` import library. It exists solely to set up runtime library search paths for CUDA and has no exported symbols needed at link time. The equivalent CMake build handles this gracefully:

```
foreach(lib c10 torch torch_cpu torch_global_deps)
  find_library(${lib}_LIB ${lib} PATHS ${LIB_TORCH_LIB_PATH} NO_DEFAULT_PATH)
  if(${lib}_LIB)
    target_link_libraries(eonclib PUBLIC ${${lib}_LIB})
  endif()
endforeach()
```

The CMake version silently skips missing libraries. The meson version treats every entry as mandatory. The fix is to add `required: false` and check `.found()`, which is the meson equivalent of the CMake pattern `[^fn:14]`.

The names themselves also fail to match. conda-forge `libmetatensor` ships `Library/lib/metatensor.dll.lib`. eOn's Windows `find_library` fallback looks for `libmetatensor / metatensor.lib`. Neither exists. The [feedstock build.bat](#) copies the DLL import lib under both names before meson setup, and does the same `lib`-prefix dance for `metatensor_torch` and `metatomic_torch`. That is a packaging alias, not a meson bug. Teach `find_library` both bare and `lib`-prefixed candidates, or stop relying on filename luck when pkg-config is present.

Cap'n Proto Code Generation and MSVC

Adding [Cap'n Proto](#) RPC support to the [v2.11.1 feedstock](#) exposed three sequential failures on Windows, each revealed only after the previous one was fixed. The serve mode integration uses [rgpot](#) as a meson subproject, which compiles `Potentials.capnp` into C++ sources at build time.

capnpc vs capnp compile

The original meson build used `capnpc` with `-o` and `c++:@OUTDIR@` as separate arguments. On Linux, `capnpc` accepted this fine. Windows `capnpc.EXE` from conda-forge did not recognize `-o` at all – not as a separate token, not concatenated. The portable alternative is the `capnp compile` subcommand:

```
# Portable (works everywhere)
capnp compile -oc++:outdir --src-prefix=srcdir schema.capnp

# Linux-only (fails on Windows capnpc.EXE)
capnpc -o c++:outdir --src-prefix=srcdir schema.capnp
```

This is not documented as a platform difference in the Cap'n Proto docs. The `capnp compile` interface is the stable one; `capnpc` appears to be a convenience wrapper with inconsistent flag parsing across platforms.

The `.c++` File Extension

`capnp compile -oc++` generates `Potentials.capnp.c++` and `Potentials.capnp.h`. GCC and Clang accept `.c++` as a valid C++ extension. MSVC's `cl.exe` does not [^fn:15]. Meson passes the file to the compiler, `cl.exe` silently ignores it, and the linker fails downstream with `LNK1181: cannot open input file 'Potentials.capnp.c++.obj'` because the object was never produced.

The fix is a thin Python wrapper that runs `capnp compile` and renames the output from `.c++` to `.cpp`. The meson `custom_target` declares the renamed extension:

```
python = find_program('python3', 'python')
capnp = find_program('capnp')

gen_rpc = custom_target(
    'gen_rpc',
    input: 'Potentials.capnp',
    output: ['Potentials.capnp.cpp', 'Potentials.capnp.h'],
    command: [python, meson.current_source_dir() / 'capnp_compile.py',
              capnp, '@OUTDIR@', meson.current_source_dir(), '@INPUT@'],
)
```

The wrapper itself is straightforward:

```
subprocess.check_call([capnp, "compile", f"-oc++:{outdir}",
                      f"--src-prefix={src_prefix}", schema])
old = os.path.join(outdir, f"{base}.capnp.c++")
new = os.path.join(outdir, f"{base}.capnp.cpp")
if os.path.exists(old):
    shutil.move(old, new)
```

Missing Transitive `ws2_32` Dependency

With code generation fixed, the build reached step [131/143] before the final link produced 22 unresolved externals:

```
kj-async.lib(async-io-win32.c++.obj) : error LNK2019: unresolved external symbol __imp_accept
kj-async.lib(async-io-win32.c++.obj) : error LNK2019: unresolved external symbol __imp_bind
kj-async.lib(async-io-win32.c++.obj) : error LNK2019: unresolved external symbol __imp_closesocket
```

Cap'n Proto's `kj-async` uses Winsock2 for async I/O on Windows. On Linux, the equivalent functions live in `libc` and need no extra linkage. The conda-forge `capnproto` package ships `kj-async.lib` as a static library; static libraries on Windows do not carry their link dependencies [^fn:16]. The consumer must link `ws2_32` explicitly.

```
_ptlrpc_extra_deps = [capnp_rpc_dep]
if host_machine.system() == 'windows'
    _ptlrpc_extra_deps += cppc.find_library('ws2_32')
endif

ptlrpc_dep = declare_dependency(
    link_with: ptlrpc,
    sources: gen_rpc,
    include_directories: _incdirs,
    dependencies: _ptlrpc_extra_deps,
)
```


Downstream consumers of `ptlrpc_dep` get `ws2_32` automatically on Windows through meson's dependency propagation. The situation is analogous to the xtb import library problem described above: the conda-forge package assumes its consumers know what system libraries to add. The fix pattern is shown in Figure 5.

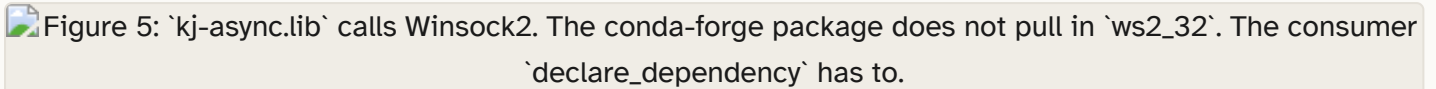
Figure 5: ``kj-async.lib`` calls Winsock2. The conda-forge package does not pull in ``ws2_32``. The consumer ``declare_dependency`` has to.

Figure 5: ``kj-async.lib`` calls Winsock2. The conda-forge package does not pull in ``ws2_32``. The consumer ``declare_dependency`` has to.

`interface` is a Windows macro

The Cap'n Proto work did not end with code generation. On recent eOn feedstock bumps (through [v2.16](#)), MSVC builds that include Cap'n Proto RPC headers fail with a cascade of template errors that mention `IPrintDialogServices` and other COM symbols. Cap'n Proto's types are fine; `windows.h` is not.

Win32 headers define `interface` as a macro (legacy COM sugar). Cap'n Proto templates use `interface` as a C++ identifier. The preprocessor rewrites the templates into nonsense before the compiler sees them. The diagnostic points at COM headers, so the first instinct is to chase `IPrintDialogServices` instead of the schema-generated code.

The feedstock `build.bat` force-includes a short guard before every translation unit:

```
#pragma once
#ifndef NOMINMAX
#define NOMINMAX
#endif
#ifdef interface
#undef interface
#endif
```

with `cl.exe` flags `/FI<path>/msvc_capnp_guard.h`, `/DNOMINMAX`, and `/DWIN32_LEAN_AND_MEAN`. The `#undef interface` line is required for a clean compile; `NOMINMAX` and `WIN32_LEAN_AND_MEAN` limit how much of the Windows header surface collides with the rest of the C++ stack. Upstream `rgpot` absorbed the earlier `fix_capnpc` / rename-to-`.cpp` logic, so the feedstock patch was dropped, but the `interface` guard remains a Windows packaging step until Cap'n Proto or the SDK stop colliding on that token.

`far` is the same family. `windef.h` still defines it as an empty macro from the 16-bit near/far pointer days. A test local named `far` becomes nothing. `#undef far` next to `#undef interface`, or do not name a variable `far`.

Flang, MSVC, and one feedstock activation

By the time in-tree Fortran returned on win-64 (feedstock [issue 15](#), EAM / GAGAFE / CuH2 and friends), conda-forge's default Fortran on Windows was `llvm-flang`, while C++ and the final link stayed on MSVC. That forms a third compiler mix next to MinGW-xtb and Cap'n Proto: flang for the legacy pots, `cl.exe` for C++, `link.exe` for the binary.

`llvm-ar` archives that `link.exe` refuses

Enabling `compiler('fortran')` in meson pulls flang activation, which steers the archiver toward `llvm-ar`. Static archives built that way (`libptlrpc`, Cap'n Proto `kj-async`, and friends) then fail the MSVC link with `LNK1107: invalid or corrupt file`. The archives are not corrupt; they use a format `link.exe` does not accept.

`build.bat` forces MSVC as the C/C++ tools and `lib.exe` as the archiver while flang still compiles the Fortran sources:

```
set "CC=cl.exe"
set "CXX=cl.exe"
set "AR=lib"
set "ARFLAGS="
set "NM=dumpbin"
set "RANLIB=:"
```

Keep `link.exe` rather than `lld-link` from the clang/flang activation so the final link and the archives share one ABI.

flang_rt and LNK1104

Flang-built objects need the flang runtime import library (`flang_rt.runtime.dynamic.lib`). It lives under the clang resource directory (`flang -print-resource-dir`), not under `%LIBRARY_LIB%` where MSVC looks by default. Without an extra `LIB=` entry, the final link fails with `LNK1104: cannot open file 'flang_rt.runtime.dynamic.lib'`.

```
for /f "delims=" %%R in ('flang -print-resource-dir 2^>nul') do (
    set "LIB=%%R\lib\x86_64-pc-windows-msvc;%%LIB%"
)
```

Stack size still needs the Meson `/STACK:16777216` (or MinGW `-Wl, - -stack,`) treatment from the GAGAFE section: enabling Fortran on Windows does not enlarge the default 1 MB stack.

Figure 6 summarizes the win-64 feedstock activation.

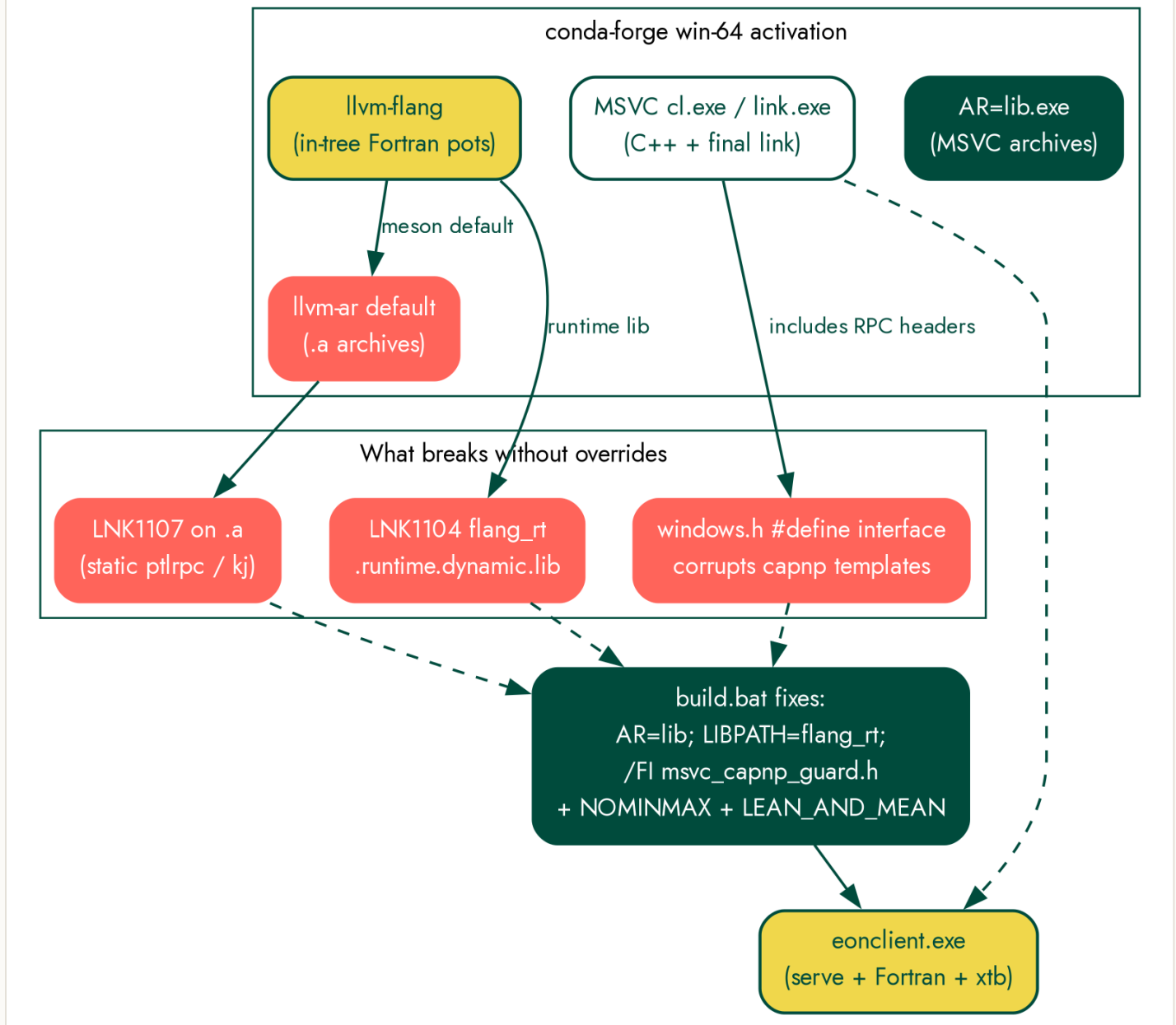


Figure 6: One feedstock activation, three failure classes: wrong archiver format, missing flang runtime path, and Win32 `interface` macro pollution of Cap'n Proto headers.

LNK2005 : two Fortran files, one potinit_

v2.14.0 statically linked aluminum and CuH2 into `eonclient`. Both defined `potinit_`. MSVC reports LNK2005. Linux `ld` is happier to pick one. The feedstock renamed the CuH2 initializer for [issue 15](#); v2.15 moved those pots behind `dlopen` and the patch went away.

Same lesson as `ws2_32`: the Windows linker is literal. Duplicate symbols are not a suggestion.

Flang and BIND(C) by-value structs

vesin's Fortran module called `vesin_neighbors`, which takes `VesinDevice` and `VesinOptions` by value. LLVM Flang on `x86_64-pc-windows-msvc` aborts:

```
not yet implemented: passing VALUE BIND(C) derived type for this target
```

That is `Optimizer/CodeGen/Target.cpp:102`. No Fortran consumer of vesin builds there at all. `rgpot` added `vesin_neighbors_byref` (pointers in, forwarded by value) and rebound the Fortran module. Upstream candidate, same pile as the other vesin patches in eOn [#389](#).

Tests that include `dlfcn.h`

The metatonic ABI test opened the engine with POSIX `dlfcn.h`. Linux and macOS CI were green. conda-forge win-64 compiled the test and stopped. The feedstock first skipped the file, then patched it, then upstream grew `DynLib` (`LoadLibraryA` / `dlopen`) in `eOn` [#376](#). That commit is on `main` and in the `v3.0.0` / `v3.1.0` tarballs, so the feedstock no longer carries the patch.

That makes the **test** portable. The production loaders (`MetatomicEngineLoader`, `XTBEngineLoader`) still `#ifndef _WIN32` and return `nullptr` on Windows. They do not call `DynLib`. A green ABI test is not `LoadLibrary` of a metatonic or xtb engine.

A unit test that only exists on POSIX is not a Windows test. It is a Linux test that pretends to be portable.

Concluding Remarks

Each failure mode here is shallow once named. Reserved filenames date to DOS. The 1 MB stack has been the Windows default for decades. The MinGW/MSVC split has been the conda-forge status quo for as long as Fortran packages have existed there. Cap'n Proto's code generation defaults (`capnpc`, `.c++` extensions) target a GCC-like compiler stack, and Win32's `interface` and `far` macros are older than Cap'n Proto itself. Mixing `llvm-flang` with `MSVC` link joins the same list: three compilers, one `build.bat`, and linker errors that read as corruption (`LNK1107`), missing files (`LNK1104`), or duplicate Fortran names (`LNK2005`). `Flang` on that target still cannot lower a by-value `BIND(C)` derived type. `MSVC` `constexpr` will not swallow a function call that GCC treats as unused. They surface when CI runs on a platform the developer does not use daily, with messages ranging from cryptic (`0xC00000FD`) to absent (`WriteConsole`) to hard linker and preprocessor stops.

What to do: add Windows to the CI matrix early, including the tests that `#include <dlfcn.h>`; watch legacy Fortran where stack allocation is the default; treat “depends on library X” as distinct from “uses the compiler stack that built library X”. When static libraries from conda-forge land on Windows, check for system libraries that shared builds would pull automatically, and check the import-lib **filename** (`metatensor.dll.lib` is not `metatensor.lib`). When Fortran and C++ share a feedstock activation, pin the archiver and runtime `LIBPATH` to the linker that produces the binary. Keep a short list of DOS device names and Win32 macros that collide with C++ identifiers within reach. Do not put `foo()` in a macro argument that expands next to `static constexpr`.

Reading list

The quantum chemistry codes and methods this port had to keep working on Windows, collected as a citation list: [5 references](#). The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.

References

Bigi, Filippo, Joseph W. Abbott, Philip Loche, Arslan Mazitov, Davide Tisi, Marcel F. Langer, Alexander Goscinski, et al. 2026. "Metatensor and Metatomic : Foundational Libraries for Interoperable Atomistic Machine Learning." **Journal of Chemical Physics** 164 (6): 64113. <<https://doi.org/10.1063/5.0304911>>.

Goswami, Rohit. 2025. "Efficient Exploration of Chemical Kinetics." October 24, 2025. <<https://doi.org/10.48550/arXiv.2510.21368>>.

Goswami, Rohit, Miha Gunde, and Hannes Jónsson. 2026. "Enhanced Climbing Image Nudged Elastic Band Method with Hessian Eigenmode Alignment." **Frontiers in Chemistry** 14 (May). <<https://doi.org/10.3389/fchem.2026.1807063>>.

Goswami, Rohit, and Hannes Jónsson. 2025. "Adaptive Pruning for Increased Robustness and Reduced Computational Overhead in Gaussian Process Accelerated Saddle Point Searches." **ChemPhysChem**, November. <<https://doi.org/10.1002/cphc.202500730>>.

Goswami, Rohit, Maxim Masterov, Satish Kamath, Alejandro Pena-Torres, and Hannes Jónsson. 2025. "Efficient Implementation of Gaussian Process Regression Accelerated Saddle Point Searches with Application to Molecular Reactions." **Journal of Chemical Theory and Computation**, July. <<https://doi.org/10.1021/acs.jctc.5c00866>>.

pretty impressive

simulations on atomic systems, combining a Python server with a C++ client. See [the documentation](<https://eondocs.org>) for details.

available:
[GPR-dimer](https://github.com/TheochemUI/gpr_sella_repro),
[OTGPD](https://github.com/TheochemUI/otgpd_repro),
[NEB-MMF](https://github.com/haozeke/nebmmf_repro),
[Metatensor](<https://github.com/metatensor/ecosystem-article>), and
[BRMS rotations](https://github.com/HaoZeke/brms_idrot_repro).

perfect sense in context, which is approximately how all of these stories begin.

`error: invalid path 'asv\_runner/aux.py'` on Windows, because the filesystem refuses to create the file.

to e0n. The format name predates my involvement by roughly two decades.

bypassing the standard I/O layer. When the handle is a file, there is no screen buffer to write to.

log](<https://github.com/TheochemUI/e0n/actions/runs/22723984535/job/65893741547>) and the quill `MacroMetadata` initializer. The logger pointer is never a constexpr operand; MSVC still walks the macro argument.

multicharge, dftd4, tblite, xtb) uses `m2w64\_c` and `m2w64\_fortran` on Windows. The Python wrappers (xtb-python, tblite-python) sidestep the problem entirely by using CFFI for runtime loading rather than link-time binding.

documentation](https://mesonbuild.com/Reference-manual_returned_compiler.html#compilerfind_library) notes that `find\_library` returns a `dependency` object. With `required: false`, the returned object's `.found()` method can be checked before use, mirroring CMake's `if(\${lib}\_LIB)` idiom.

[recognizes](<https://learn.microsoft.com/en-us/cpp/build/reference/tc-tp-tc-tp-specify-source-file-type>) `.c`, `.cpp`, `.cxx`, and `.cc` by default. The `/Tp` flag can force a file to be treated as C++, but meson does not inject it for unknown extensions.

in `DT\_NEEDED` entries, resolved at load time. Static libraries (`.a`) on both platforms are just archives of object files -- but the Linux linker pulls symbols transitively from shared deps while the Windows linker requires every static dependency to be listed explicitly.